ik wil met deze data set in python jupiter sheet visualisaties gaan maken, kan jij mij daarbij zo helpen?

31 Mar

Ran 2 commands, viewed a file

Goed, ik heb de dataset goed bekeken! Het gaat om een systematische review van biobased verpakkingsmaterialen met 222 artikelen (82 included). Nu zet ik een kant-en-klaar Jupyter Notebook voor je op met visualisaties die passen bij deze data.

Ran a command, created a file, read a file

Het notebook staat klaar! Zet het `.ipynb`-bestand in dezelfde map als je Excel-bestand en open het in Jupyter. Het bevat **14 visualisaties** verdeeld over de dataset:

Screening & publicaties

- Included vs. excluded (taartdiagram)
- Exclusieredenen (staafdiagram)
- Publicatiejaar van included studies

Materialen

- Polymeertype (horizontale staafdiagram)
- Materiaalfamilie (taartdiagram)
- Verwerkingsmethoden
- Additieven

Barrière-eigenschappen

- Filmdikte per polymeertype (boxplot)
- WVP per polymeertype (stripplot op log-schaal)
- OTR vs. WVP scatterplot per materiaalfamilie
- Heatmap van missende barrière-waarden per artikel
- WVP biobased vs. referentiepolymeer

Duurzaamheid

- Biobased & Biodegradable (gecombineerd)
- Voedselcontact getest?

Enige tip: de dataset heeft relatief weinig gevulde waarden voor OTR en PO_2 (11 resp. 10 stuks), dus die plots werken het beste bij WVP. Wil je aanpassingen, extra plots of andere kleuren?

Thesisvisualisaties sharondebondt
IPYNB

kan je van de tabel in de sheet conversion factors in word een leuke tabel maken voor mijn rapport?

31 Mar

Laat me eerst de sheet bekijken.

Ran 2 commands, viewed a file

Nu ga ik de tabel opbouwen. De data heeft veel kolommen — ik maak een nette, overzichtelijke tabel met de meest relevante info (eigenschap, van-eenheid, naar-eenheid, en de conversiefactoren per dimensie).

Ran 5 commands, created a file, read a file

Het Word-document staat klaar! De tabel is in liggende A4-indeling gezet zodat alle kolommen goed passen. De opmaak:

- **Donkergroene header** met witte tekst voor de kolomtitels
- **Groene groepsbalk** per barrière-eigenschap (WVP, WVTR, PO_2, OTR, PCO_2)
- **Afwisselende witte/lichtgroene rijen** voor leesbaarheid
- **8 kolommen**: Property · From unit · To unit · Massa/vol. · Dikte · Oppervlak · Tijd · Druk
- Een voetnoot onder de tabel met uitleg over de conversiemethode

Wil je iets aanpassen, zoals kleuren, lettertype, staande A4, of een andere tabelopbouw?

Conversionfactors table
Document · DOCX

oke ik wil nu weer verder in jupiter. ik wil heatmaps met alle combinaties van rh en temperaturen. maak 6 rijen voor wvtr, otr, co2tr voor biobased or biodegradable materials en in de columns de mogelijke cominaties. in de heat map wil ik dat je benoemt hoe veel metingen het zijn per combinatie

31 Mar

Ran 3 commands

Goed, ik heb alles wat ik nodig heb. De 6 rijen zijn: WVTR/OTR/CO2TR × Biobased/Biodegradable. Nu maak ik het notebook-cel aan.

Jupyter notebook cell with heatmaps

Claude's response was interrupted

sorry ik bedoel ook de reference materialen erbij

31 Mar

Ran 3 commands

Duidelijk. De referentiematerialen zijn conventionele polymeren (geen biobased/biodegradable label) — de 6 rijen worden dus **WVTR / OTR / CO_2TR × Biobased materiaal / Referentiemateriaal**. Nu maak ik het notebook.

Created 2 files, ran a command, read a file

Het notebook staat klaar! Zet het in dezelfde map als je Excel-bestand. De opbouw:

3 subplots (WVTR / OTR / CO_2TR), elk met 2 rijen:

- 🟩 **Groen** — Biobased / Biodegradable metingen
- 🟦 **Blauw** — Referentiematerialen (conventionele polymeren)

De kolommen zijn alle unieke T–RH combinaties die in de data voorkomen (gesorteerd op temperatuur dan RH). Per cel staat het **aantal metingen** als getal, cellen zonder data krijgen een streepje. Rechts staat het totaal `n =` per rij.

Wat je in de data kunt verwachten:

- **WVTR** heeft de meeste variatie: 4 temperaturen (20–30°C) en 7 RH-waarden
- **OTR en CO_2TR** zijn compacter (23–25°C, voornamelijk 0% en 50–53% RH)

Heatmaps txrh
IPYNB

kan je me niet hier de code geven zodat ik m kan kopieren en plakken?

31 Mar

Natuurlijk! Hier zijn de drie cellen apart zodat je ze makkelijk kunt kopiëren:

Cel 1 – Imports & data inladen

python

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import matplotlib.patches as mpatches
import warnings
warnings.filterwarnings('ignore')

# ── Data inladen ──────────────────────────────────────────────────────────────
FILE = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df   = pd.read_excel(FILE, sheet_name='Data')
bio  = df[(df['Biobased'] == 'Yes') | (df['Biodegradable'] == 'Yes')].copy()

print(f"Totaal rijen dataset:         {len(df)}")
print(f"Biobased / Biodegradable:     {len(bio)}")
```

Cel 2 – Configuratie & hulpfuncties

python

```python
# ── Configuratie per property ─────────────────────────────────────────────────
PROPS = [
    (
        'WVTR',
        'WVTR100_g·m-2·d-1 ',          'WVTR_temperature_°C',       'WVTR_RH_%',
        'Reference_WVTR100_g·m-2·d-1 ', 'Reference_WVTR_T_°C',       'Reference_WVTR_RH_%',
    ),
    (
        'OTR',
        'OTR100_cm3·m-2·d-1·bar-1',              'OTR_temperature_°C',        'OTR_RH_%',
        'Reference_OTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',    'Reference_OTR_temperature°C','Reference_OTR_RH_%',
    ),
    (
        'CO₂TR',
        'CO2TR100_cm3·m-2·d-1·bar-1',            'CO2TR_temperature_°C',      'CO2TR_RH_%',
        'Reference_COTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',    'Reference_COTR_temperature°C','Reference_COTR_RH_%',
    ),
]

CMAP_BIO = mcolors.LinearSegmentedColormap.from_list('bio', ['#EEF7EE', '#66BB6A', '#1B5E20'])
CMAP_REF = mcolors.LinearSegmentedColormap.from_list('ref', ['#EEF3FB', '#64B5F6', '#0D47A1'])

def count_combos(source_df, val_col, t_col, rh_col):
    sub = source_df[[val_col, t_col, rh_col]].dropna().copy()
    sub['T']  = sub[t_col].round(1)
    sub['RH'] = sub[rh_col].round(1)
    return sub.groupby(['T', 'RH']).size().reset_index(name='n')

def col_label(t, rh):
    t_s  = str(int(t))  if t  == int(t)  else str(t)
    rh_s = str(int(rh)) if rh == int(rh) else str(rh)
    return f"{t_s} °C\n{rh_s} % RH"

print("Configuratie gereed ✓")
```

Cel 3 – Heatmaps

python

```python
fig, axes = plt.subplots(3, 1, figsize=(17, 10), constrained_layout=True)
```

```
ROW_INFO = [
    ('Biobased /\nBiodegradable', CMAP_BIO, '#1B5E20'),
    ('Referentie-\nmateriaal',    CMAP_REF, '#0D47A1'),
]

for ax, (label, m_val, m_t, m_rh, r_val, r_t, r_rh) in zip(axes, PROPS):

    bio_cnt = count_combos(bio, m_val, m_t, m_rh)
    ref_cnt = count_combos(df,  r_val, r_t, r_rh)

    combos = (
        pd.concat([bio_cnt[['T','RH']], ref_cnt[['T','RH']]])
        .drop_duplicates()
        .sort_values(['T','RH'])
        .reset_index(drop=True)
    )
    n_cols = len(combos)

    matrix = np.zeros((2, n_cols), dtype=int)
    for i, (t, rh) in enumerate(zip(combos['T'], combos['RH'])):
        b = bio_cnt[(bio_cnt['T']==t) & (bio_cnt['RH']==rh)]
        r = ref_cnt[(ref_cnt['T']==t) & (ref_cnt['RH']==rh)]
        if not b.empty: matrix[0, i] = b['n'].values[0]
        if not r.empty: matrix[1, i] = r['n'].values[0]

    vmax = max(matrix.max(), 1)

    for row_i, (_, cmap, txt_dark) in enumerate(ROW_INFO):
        for col_i in range(n_cols):
            val   = matrix[row_i, col_i]
            color = cmap(val / vmax)

            rect = plt.Rectangle(
                [col_i - 0.5, row_i - 0.5], 1, 1,
                fc=color, ec='white', lw=2, zorder=1
            )
            ax.add_patch(rect)

            if val > 0:
                brightness = 0.299*color[0] + 0.587*color[1] + 0.114*color[2]
                fc = 'white' if brightness < 0.52 else txt_dark
                ax.text(col_i, row_i, str(val),
                        ha='center', va='center',
                        fontsize=13, fontweight='bold', color=fc, zorder=2)
            else:
                ax.text(col_i, row_i, '–',
                        ha='center', va='center',
                        fontsize=12, color='#C8C8C8', zorder=2)

    for row_i, (_, _, clr) in enumerate(ROW_INFO):
        total = int(matrix[row_i].sum())
        ax.text(n_cols - 0.4, row_i, f" n = {total}",
                va='center', ha='left', fontsize=10.5,
                fontweight='bold', color=clr)

    ax.set_xlim(-0.5, n_cols + 0.7)
    ax.set_ylim(-0.5, 1.5)
    ax.set_xticks(range(n_cols))
    ax.set_xticklabels([col_label(t, rh) for t, rh in zip(combos['T'], combos['RH'])],
                       fontsize=9, ha='center')
    ax.xaxis.set_tick_params(length=0)
    ax.set_yticks([0, 1])
    ax.set_yticklabels([r[0] for r in ROW_INFO], fontsize=10, fontweight='bold')
    ax.yaxis.set_tick_params(length=0)
    ax.set_title(f"{label}  –  aantal metingen per T × RH combinatie",
                 fontsize=12, fontweight='bold', loc='left', pad=8)
    for spine in ax.spines.values():
        spine.set_visible(False)
    ax.set_facecolor('#F7F7F7')

patches = [
    mpatches.Patch(color='#2E7D32', label='Biobased / Biodegradable materiaal'),
    mpatches.Patch(color='#1565C0', label='Referentiemateriaal (conventioneel polymeer)'),
]
fig.legend(handles=patches, loc='lower center', ncol=2,
           fontsize=10, frameon=False, bbox_to_anchor=(0.5, -0.03))
fig.suptitle('Verdeling van meetcondities (T × RH) per barrière-eigenschap',
             fontsize=14, fontweight='bold')

plt.savefig('heatmaps_TxRH.png', dpi=150, bbox_inches='tight')
plt.show()
```

Plak elke cel in een aparte Jupyter-cel. Zorg dat het Excel-bestand in dezelfde map staat als je notebook!

ik heb een keer dit gemaakt in gemini, kun je hetzelfde doen voor dit bestand, en voeg ook het aantal metingen toe van otr, co2tr en wvtr voor de reference materials: import pandas as pd import matplotlib.pyplot as plt import matplotlib.ticker as ticker import seaborn as sns import os # 1. Load the dataset file_name = 'ThesisSharondeBondt_SystematicReview_CleanedData_v1.xlsx' df = pd.read_excel(file_name) # 2. Define the columns and counts target_cols = { 'OTR': 'OTR_norm_100µm', 'CO2TR': 'CO2TR_norm_100µm', 'WVTR': 'WVTR_norm_100µm' } total_papers = df['Article_ID'].nunique() total_rows = len(df) paper_stats = {label: df[df[col].notna()]['Article_ID'].nunique() for label, col in target_cols.items() if col in df.columns} measure_stats = {label: df[col].notna().sum() for label, col in target_cols.items() if col in df.columns} paper_series = pd.Series(paper_stats).sort_values(ascending=False) measure_series = pd.Series(measure_stats).sort_values(ascending=False) # 3. Setup Plotting sns.set_theme(style="whitegrid") fig, axes = plt.subplots(1, 2, figsize=(15, 7)) def add_labels(ax, total, total_label): for p in ax.patches: count = int(p.get_height()) percentage = (count / total) * 100 if total > 0 else 0 ax.annotate(f'{count}\n({percentage:.1f}%)', (p.get_x() + p.get_width() / 2., p.get_height()), ha='center', va='bottom', xytext=(0, 8), textcoords='offset points',

31 Mar

Eerst even de juiste kolomnamen checken in het nieuwe bestand:

Ran 2 commands

Alles duidelijk. Hier is de code:

python

```python
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import os

# 1. Load the dataset
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')

# 2. Define columns
meas_cols = {
   'OTR':   'OTR100_cm3·m-2·d-1·bar-1',
   'CO₂TR': 'CO2TR100_cm3·m-2·d-1·bar-1',
   'WVTR':  'WVTR100_g·m-2·d-1 ',
}
ref_cols = {
   'OTR':   'Reference_OTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
   'CO₂TR': 'Reference_COTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
   'WVTR':  'Reference_WVTR100_g·m-2·d-1 ',
}

total_papers = df['Article_ID'].nunique()
total_rows   = len(df)

# 3. Build stats — measurements + reference side by side
paper_data   = {}
measure_data = {}

for label in meas_cols:
   mc = meas_cols[label]
   rc = ref_cols[label]
   paper_data[label]   = {
      'Biobased / Biodegradable': df[df[mc].notna()]['Article_ID'].nunique(),
      'Reference material':       df[df[rc].notna()]['Article_ID'].nunique(),
   }
   measure_data[label] = {
      'Biobased / Biodegradable': int(df[mc].notna().sum()),
      'Reference material':       int(df[rc].notna().sum()),
   }

paper_df   = pd.DataFrame(paper_data).T    # rows = OTR/CO₂TR/WVTR
measure_df = pd.DataFrame(measure_data).T

# 4. Plot
sns.set_theme(style='whitegrid')
PALETTE = {'Biobased / Biodegradable': '#2E7D32', 'Reference material': '#1565C0'}

fig, axes = plt.subplots(1, 2, figsize=(15, 7))

def grouped_bars(ax, data_df, ylabel, total, total_label):
```

```
    x     = range(len(data_df))
    width  = 0.35
    groups = data_df.columns.tolist()

    for i, group in enumerate(groups):
        offset = (i - 0.5) * width
        bars   = ax.bar(
            [xi + offset for xi in x],
            data_df[group],
            width=width,
            label=group,
            color=PALETTE[group],
            edgecolor='white',
            linewidth=0.8,
        )
        for bar in bars:
            count = int(bar.get_height())
            if count == 0:
                continue
            pct = (count / total) * 100
            ax.annotate(
                f'{count}\n({pct:.1f}%)',
                (bar.get_x() + bar.get_width() / 2, bar.get_height()),
                ha='center', va='bottom',
                xytext=(0, 6), textcoords='offset points',
                fontsize=10, fontweight='bold',
                color=PALETTE[group],
            )

    ax.set_xticks(list(x))
    ax.set_xticklabels(data_df.index, fontsize=12)
    ax.set_ylabel(ylabel, fontsize=12)
    ax.set_xlabel('Barrier property', fontsize=12)
    ax.yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
    ax.set_ylim(0, ax.get_ylim()[1] * 1.3)
    ax.legend(fontsize=10, frameon=False)
    ax.text(0.5, -0.13, f'Total {total_label} in dataset: {total}',
            ha='center', transform=ax.transAxes,
            fontsize=10, style='italic', color='#555555')

grouped_bars(axes[0], paper_df,   'Number of papers',       total_papers, 'papers')
grouped_bars(axes[1], measure_df, 'Number of measurements', total_rows,   'measurements')

axes[0].set_title('Representation by Unique Articles',     fontsize=15, pad=20)
axes[1].set_title('Representation by Total Measurements',  fontsize=15, pad=20)

plt.tight_layout()

# 5. Save
output_folder = 'visualizations'
os.makedirs(output_folder, exist_ok=True)
save_path = os.path.join(output_folder, 'barrier_property_representativeness.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
print(f"Done! Saved as '{save_path}'.")
plt.show()
```

De belangrijkste aanpassingen ten opzichte van het Gemini-script:

- **Gegroepeerde staafdiagrammen** per property — groen voor biobased/biodegradable, blauw voor referentiematerialen
- Juiste kolomnamen voor het nieuwe bestand (`sheet_name='Data'`, unicode-kolomnamen)
- Percentages worden berekend t.o.v. het totaal in de dataset, net als in het origineel

het font mag trouwens in times new roman en dit bericht krijg ik: C:\Users\Sharon de Bondt\AppData\Local\Temp\ipykernel_22816\3670247902.py:98: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial. plt.tight_layout() C:\Users\Sharon de Bondt\AppData\Local\Temp\ipykernel_22816\3670247902.py:104: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial. plt.savefig(save_path, dpi=300, bbox_inches='tight') C:\anaconda\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial. fig.canvas.print_figure(bytes_io, **kw)

31 Mar

De ₂ in CO_2TR is het probleem — Times New Roman heeft dat subscript-teken niet. De oplossing is om `CO$_2$TR` te gebruiken (LaTeX-notatie van matplotlib), dan rendert het correct. Hier is de aangepaste code:

python

```python
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import os

# 1. Load the dataset
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')

# 2. Font instellen op Times New Roman
plt.rcParams['font.family'] = 'Times New Roman'

# 3. Define columns  (CO2TR zonder subscript-unicode)
meas_cols = {
    'OTR':        'OTR100_cm3·m-2·d-1·bar-1',
    'CO$_2$TR':    'CO2TR100_cm3·m-2·d-1·bar-1',
    'WVTR':       'WVTR100_g·m-2·d-1 ',
}
ref_cols = {
    'OTR':        'Reference_OTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'CO$_2$TR':    'Reference_COTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'WVTR':       'Reference_WVTR100_g·m-2·d-1 ',
}

total_papers = df['Article_ID'].nunique()
total_rows   = len(df)

# 4. Build stats
paper_data   = {}
measure_data = {}

for label in meas_cols:
    mc = meas_cols[label]
    rc = ref_cols[label]
    paper_data[label] = {
        'Biobased / Biodegradable': df[df[mc].notna()]['Article_ID'].nunique(),
        'Reference material':       df[df[rc].notna()]['Article_ID'].nunique(),
    }
    measure_data[label] = {
        'Biobased / Biodegradable': int(df[mc].notna().sum()),
        'Reference material':       int(df[rc].notna().sum()),
    }

paper_df   = pd.DataFrame(paper_data).T
measure_df = pd.DataFrame(measure_data).T

# 5. Plot
sns.set_theme(style='whitegrid')
plt.rcParams['font.family'] = 'Times New Roman'   # herhalen na set_theme (reset)

PALETTE = {'Biobased / Biodegradable': '#2E7D32', 'Reference material': '#1565C0'}

fig, axes = plt.subplots(1, 2, figsize=(15, 7))

def grouped_bars(ax, data_df, ylabel, total, total_label):
    x     = range(len(data_df))
    width  = 0.35
    groups = data_df.columns.tolist()

    for i, group in enumerate(groups):
        offset = (i - 0.5) * width
        bars   = ax.bar(
            [xi + offset for xi in x],
            data_df[group],
            width=width,
            label=group,
            color=PALETTE[group],
            edgecolor='white',
            linewidth=0.8,
        )
        for bar in bars:
            count = int(bar.get_height())
            if count == 0:
                continue
            pct = (count / total) * 100
            ax.annotate(
                f'{count}\n({pct:.1f}%)',
                (bar.get_x() + bar.get_width() / 2, bar.get_height()),
                ha='center', va='bottom',
                xytext=(0, 6), textcoords='offset points',
```

```
            fontsize=10, fontweight='bold',
            color=PALETTE[group],
        )

    ax.set_xticks(list(x))
    ax.set_xticklabels(data_df.index, fontsize=12)
    ax.set_ylabel(ylabel, fontsize=12)
    ax.set_xlabel('Barrier property', fontsize=12)
    ax.yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
    ax.set_ylim(0, ax.get_ylim()[1] * 1.3)
    ax.legend(fontsize=10, frameon=False)
    ax.text(0.5, -0.13, f'Total {total_label} in dataset: {total}',
        ha='center', transform=ax.transAxes,
        fontsize=10, style='italic', color='#555555')

grouped_bars(axes[0], paper_df,   'Number of papers',       total_papers, 'papers')
grouped_bars(axes[1], measure_df, 'Number of measurements', total_rows,   'measurements')

axes[0].set_title('Representation by Unique Articles',    fontsize=15, pad=20)
axes[1].set_title('Representation by Total Measurements', fontsize=15, pad=20)

plt.tight_layout()

# 6. Save
output_folder = 'visualizations'
os.makedirs(output_folder, exist_ok=True)
save_path = os.path.join(output_folder, 'barrier_property_representativeness.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
print(f"Done! Saved as '{save_path}'.")
plt.show()
```

De twee wijzigingen:

- `plt.rcParams['font.family'] = 'Times New Roman'` — staat nu **twee keer**: eenmaal voor en eenmaal na `sns.set_theme()`, omdat seaborn de rcParams reset
- `CO$_2$TR` in plaats van `CO₂TR` — matplotlib rendert dit als echte subscript zonder dat het font het teken hoeft te ondersteunen

kunnen de figuren weer worden opgeslagen in visualisations_v7?

31 Mar

Alleen de mapnaam aanpassen:

python

```
output_folder = 'visualizations_v7'
```

Dus regel 6 van het laatste blok wordt:

python

```
# 6. Save
output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)
save_path = os.path.join(output_folder, 'barrier_property_representativeness.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
print(f"Done! Saved as '{save_path}'.")
plt.show()
```

De rest van de code blijft hetzelfde.

oja het moet natuurlijk wel gefilterd worden op de papers die included zijn na screening :)

31 Mar

Voeg dan één filterregel toe na het inladen van de data:

python

```python
# 1. Load the dataset
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')

# Filter op included papers
df = df[df['Included_after_screening'] == 'Yes'].copy()
```

Alle berekeningen daarna werken automatisch alleen op de included papers. Wel even checken of de kolomnaam exact klopt — in de cleaned dataset heette die kolom `Included_after_screening`, maar je kunt dit snel verifiëren met:

python

```python
print(df.columns.tolist())
```

ja en de reference materials en biobased materials dus in aparte figuren aub

31 Mar

Twee aparte figuren dus. Hier is de volledige aangepaste code:

python

```python
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import os

# 1. Load & filter
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')
df = df[df['Included_after_screening'] == 'Yes'].copy()

# 2. Font
plt.rcParams['font.family'] = 'Times New Roman'

# 3. Columns
meas_cols = {
    'OTR':     'OTR100_cm3·m-2·d-1·bar-1',
    'CO$_2$TR': 'CO2TR100_cm3·m-2·d-1·bar-1',
    'WVTR':    'WVTR100_g·m-2·d-1 ',
}
ref_cols = {
    'OTR':     'Reference_OTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'CO$_2$TR': 'Reference_COTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'WVTR':    'Reference_WVTR100_g·m-2·d-1 ',
}

total_papers = df['Article_ID'].nunique()
total_rows   = len(df)

# 4. Stats
bio_paper_stats   = {l: df[df[meas_cols[l]].notna()]['Article_ID'].nunique() for l in meas_cols}
bio_measure_stats = {l: int(df[meas_cols[l]].notna().sum()) for l in meas_cols}
ref_paper_stats   = {l: df[df[ref_cols[l]].notna()]['Article_ID'].nunique() for l in ref_cols}
ref_measure_stats = {l: int(df[ref_cols[l]].notna().sum()) for l in ref_cols}

bio_paper_series   = pd.Series(bio_paper_stats).sort_values(ascending=False)
bio_measure_series = pd.Series(bio_measure_stats).sort_values(ascending=False)
ref_paper_series   = pd.Series(ref_paper_stats).sort_values(ascending=False)
ref_measure_series = pd.Series(ref_measure_stats).sort_values(ascending=False)

# 5. Helper
def add_labels(ax, total, total_label):
    for p in ax.patches:
        count = int(p.get_height())
        if count == 0:
            continue
        pct = (count / total) * 100
        ax.annotate(
            f'{count}\n({pct:.1f}%)',
```

```
        (p.get_x() + p.get_width() / 2, p.get_height()),
        ha='center', va='bottom',
        xytext=(0, 8), textcoords='offset points',
        fontsize=11, fontweight='bold'
    )
ax.set_ylim(0, max(ax.get_ylim()[1] * 1.25, 10))
ax.text(0.5, -0.15, f'Total {total_label} in dataset: {total}',
        ha='center', transform=ax.transAxes,
        fontsize=10, style='italic', color='#555555')

output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)

# ── Figuur 1: Biobased / Biodegradable materials ──────────────────────────
sns.set_theme(style='whitegrid')
plt.rcParams['font.family'] = 'Times New Roman'

fig1, axes1 = plt.subplots(1, 2, figsize=(15, 7))
fig1.suptitle('Biobased / Biodegradable Materials', fontsize=16, fontweight='bold', y=1.02)

sns.barplot(x=bio_paper_series.index, y=bio_paper_series.values,
        ax=axes1[0], palette='viridis',
        hue=bio_paper_series.index, legend=False)
axes1[0].set_title('Representation by Unique Articles', fontsize=15, pad=25)
axes1[0].set_ylabel('Number of Papers', fontsize=12)
axes1[0].set_xlabel('Barrier Property', fontsize=12)
axes1[0].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes1[0], total_papers, 'papers')

sns.barplot(x=bio_measure_series.index, y=bio_measure_series.values,
        ax=axes1[1], palette='magma',
        hue=bio_measure_series.index, legend=False)
axes1[1].set_title('Representation by Total Measurements', fontsize=15, pad=25)
axes1[1].set_ylabel('Number of Measurements', fontsize=12)
axes1[1].set_xlabel('Barrier Property', fontsize=12)
axes1[1].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes1[1], total_rows, 'measurements')

plt.tight_layout()
path1 = os.path.join(output_folder, 'barrier_representativeness_biobased.png')
plt.savefig(path1, dpi=300, bbox_inches='tight')
print(f"Saved: '{path1}'")
plt.show()

# ── Figuur 2: Reference materials ──────────────────────────────────────────
sns.set_theme(style='whitegrid')
plt.rcParams['font.family'] = 'Times New Roman'

fig2, axes2 = plt.subplots(1, 2, figsize=(15, 7))
fig2.suptitle('Reference Materials', fontsize=16, fontweight='bold', y=1.02)

sns.barplot(x=ref_paper_series.index, y=ref_paper_series.values,
        ax=axes2[0], palette='viridis',
        hue=ref_paper_series.index, legend=False)
axes2[0].set_title('Representation by Unique Articles', fontsize=15, pad=25)
axes2[0].set_ylabel('Number of Papers', fontsize=12)
axes2[0].set_xlabel('Barrier Property', fontsize=12)
axes2[0].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes2[0], total_papers, 'papers')

sns.barplot(x=ref_measure_series.index, y=ref_measure_series.values,
        ax=axes2[1], palette='magma',
        hue=ref_measure_series.index, legend=False)
axes2[1].set_title('Representation by Total Measurements', fontsize=15, pad=25)
axes2[1].set_ylabel('Number of Measurements', fontsize=12)
axes2[1].set_xlabel('Barrier Property', fontsize=12)
axes2[1].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes2[1], total_rows, 'measurements')

plt.tight_layout()
path2 = os.path.join(output_folder, 'barrier_representativeness_reference.png')
plt.savefig(path2, dpi=300, bbox_inches='tight')
print(f"Saved: '{path2}'")
plt.show()
```

De twee figuren worden opgeslagen als:

- `visualizations_v7/barrier_representativeness_biobased.png`
- `visualizations_v7/barrier_representativeness_reference.png`

ik wil niet dat het totaal aan measurements dat van reference + biomaterials is maar apart van elkaar

31 Mar

Kleine aanpassing — de totalen apart berekenen en meegeven aan `add_labels`:

python

```python
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import os

# 1. Load & filter
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')
df = df[df['Included_after_screening'] == 'Yes'].copy()

# 2. Font
plt.rcParams['font.family'] = 'Times New Roman'

# 3. Columns
meas_cols = {
    'OTR':     'OTR100_cm3·m-2·d-1·bar-1',
    'CO$_2$TR': 'CO2TR100_cm3·m-2·d-1·bar-1',
    'WVTR':    'WVTR100_g·m-2·d-1 ',
}
ref_cols = {
    'OTR':     'Reference_OTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'CO$_2$TR': 'Reference_COTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'WVTR':    'Reference_WVTR100_g·m-2·d-1 ',
}

total_papers = df['Article_ID'].nunique()

# Totalen apart
total_bio_measurements = sum(int(df[c].notna().sum()) for c in meas_cols.values())
total_ref_measurements = sum(int(df[c].notna().sum()) for c in ref_cols.values())

# 4. Stats
bio_paper_series   = pd.Series({l: df[df[meas_cols[l]].notna()]['Article_ID'].nunique() for l in meas_cols}).sort_values(ascending=False)
bio_measure_series = pd.Series({l: int(df[meas_cols[l]].notna().sum()) for l in meas_cols}).sort_values(ascending=False)
ref_paper_series   = pd.Series({l: df[df[ref_cols[l]].notna()]['Article_ID'].nunique() for l in ref_cols}).sort_values(ascending=False)
ref_measure_series = pd.Series({l: int(df[ref_cols[l]].notna().sum()) for l in ref_cols}).sort_values(ascending=False)

# 5. Helper
def add_labels(ax, total, total_label):
    for p in ax.patches:
        count = int(p.get_height())
        if count == 0:
            continue
        pct = (count / total) * 100
        ax.annotate(
            f'{count}\n({pct:.1f}%)',
            (p.get_x() + p.get_width() / 2, p.get_height()),
            ha='center', va='bottom',
            xytext=(0, 8), textcoords='offset points',
            fontsize=11, fontweight='bold'
        )
    ax.set_ylim(0, max(ax.get_ylim()[1] * 1.25, 10))
    ax.text(0.5, -0.15, f'Total {total_label} in dataset: {total}',
            ha='center', transform=ax.transAxes,
            fontsize=10, style='italic', color='#555555')

output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)

# ── Figuur 1: Biobased / Biodegradable materials ──────────────────────────
sns.set_theme(style='whitegrid')
plt.rcParams['font.family'] = 'Times New Roman'

fig1, axes1 = plt.subplots(1, 2, figsize=(15, 7))
fig1.suptitle('Biobased / Biodegradable Materials', fontsize=16, fontweight='bold', y=1.02)

sns.barplot(x=bio_paper_series.index, y=bio_paper_series.values,
            ax=axes1[0], palette='viridis', hue=bio_paper_series.index, legend=False)
```

```
axes1[0].set_title('Representation by Unique Articles', fontsize=15, pad=25)
axes1[0].set_ylabel('Number of Papers', fontsize=12)
axes1[0].set_xlabel('Barrier Property', fontsize=12)
axes1[0].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes1[0], total_papers, 'papers')

sns.barplot(x=bio_measure_series.index, y=bio_measure_series.values,
            ax=axes1[1], palette='magma', hue=bio_measure_series.index, legend=False)
axes1[1].set_title('Representation by Total Measurements', fontsize=15, pad=25)
axes1[1].set_ylabel('Number of Measurements', fontsize=12)
axes1[1].set_xlabel('Barrier Property', fontsize=12)
axes1[1].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes1[1], total_bio_measurements, 'measurements')

plt.tight_layout()
path1 = os.path.join(output_folder, 'barrier_representativeness_biobased.png')
plt.savefig(path1, dpi=300, bbox_inches='tight')
print(f"Saved: '{path1}'")
plt.show()

# ── Figuur 2: Reference materials ──────────────────────────────────────────
sns.set_theme(style='whitegrid')
plt.rcParams['font.family'] = 'Times New Roman'

fig2, axes2 = plt.subplots(1, 2, figsize=(15, 7))
fig2.suptitle('Reference Materials', fontsize=16, fontweight='bold', y=1.02)

sns.barplot(x=ref_paper_series.index, y=ref_paper_series.values,
            ax=axes2[0], palette='viridis', hue=ref_paper_series.index, legend=False)
axes2[0].set_title('Representation by Unique Articles', fontsize=15, pad=25)
axes2[0].set_ylabel('Number of Papers', fontsize=12)
axes2[0].set_xlabel('Barrier Property', fontsize=12)
axes2[0].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes2[0], total_papers, 'papers')

sns.barplot(x=ref_measure_series.index, y=ref_measure_series.values,
            ax=axes2[1], palette='magma', hue=ref_measure_series.index, legend=False)
axes2[1].set_title('Representation by Total Measurements', fontsize=15, pad=25)
axes2[1].set_ylabel('Number of Measurements', fontsize=12)
axes2[1].set_xlabel('Barrier Property', fontsize=12)
axes2[1].yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
add_labels(axes2[1], total_ref_measurements, 'measurements')

plt.tight_layout()
path2 = os.path.join(output_folder, 'barrier_representativeness_reference.png')
plt.savefig(path2, dpi=300, bbox_inches='tight')
print(f"Saved: '{path2}'")
plt.show()
```

De enige wijziging: `total_bio_measurements` en `total_ref_measurements` worden nu apart berekend (som van OTR + CO_2TR + WVTR per groep) en elk meegegeven aan de bijbehorende figuur.

ik zou ook graag willen dat het van papers en aantal measurements apart van elkaar opgeslagen wordt zodat ik zelf de grootte individueel kan aanpassen in word

31 Mar

Kleine aanpassing — elk subplot apart opslaan:

python

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import os

# 1. Load & filter
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')
df = df[df['Included_after_screening'] == 'Yes'].copy()

# 2. Font
plt.rcParams['font.family'] = 'Times New Roman'
```

```python
# 3. Columns
meas_cols = {
    'OTR':     'OTR100_cm3·m-2·d-1·bar-1',
    'CO$_2$TR': 'CO2TR100_cm3·m-2·d-1·bar-1',
    'WVTR':    'WVTR100_g·m-2·d-1 ',
}
ref_cols = {
    'OTR':     'Reference_OTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'CO$_2$TR': 'Reference_COTR100_cm³·m⁻²·d⁻¹·bar⁻¹ ',
    'WVTR':    'Reference_WVTR100_g·m-2·d-1 ',
}

total_papers           = df['Article_ID'].nunique()
total_bio_measurements = sum(int(df[c].notna().sum()) for c in meas_cols.values())
total_ref_measurements = sum(int(df[c].notna().sum()) for c in ref_cols.values())

# 4. Stats
bio_paper_series   = pd.Series({l: df[df[meas_cols[l]].notna()]['Article_ID'].nunique() for l in meas_cols}).sort_values(ascending=False)
bio_measure_series = pd.Series({l: int(df[meas_cols[l]].notna().sum()) for l in meas_cols}).sort_values(ascending=False)
ref_paper_series   = pd.Series({l: df[df[ref_cols[l]].notna()]['Article_ID'].nunique() for l in ref_cols}).sort_values(ascending=False)
ref_measure_series = pd.Series({l: int(df[ref_cols[l]].notna().sum()) for l in ref_cols}).sort_values(ascending=False)

# 5. Helper
def add_labels(ax, total, total_label):
    for p in ax.patches:
        count = int(p.get_height())
        if count == 0:
            continue
        pct = (count / total) * 100
        ax.annotate(
            f'{count}\n({pct:.1f}%)',
            (p.get_x() + p.get_width() / 2, p.get_height()),
            ha='center', va='bottom',
            xytext=(0, 8), textcoords='offset points',
            fontsize=11, fontweight='bold'
        )
    ax.set_ylim(0, max(ax.get_ylim()[1] * 1.25, 10))
    ax.text(0.5, -0.15, f'Total {total_label} in dataset: {total}',
            ha='center', transform=ax.transAxes,
            fontsize=10, style='italic', color='#555555')

def save_chart(series, ylabel, title, total, total_label, palette, filename):
    sns.set_theme(style='whitegrid')
    plt.rcParams['font.family'] = 'Times New Roman'
    fig, ax = plt.subplots(figsize=(7, 6))
    sns.barplot(x=series.index, y=series.values, ax=ax,
                palette=palette, hue=series.index, legend=False)
    ax.set_title(title, fontsize=15, pad=25)
    ax.set_ylabel(ylabel, fontsize=12)
    ax.set_xlabel('Barrier Property', fontsize=12)
    ax.yaxis.set_major_locator(ticker.MaxNLocator(integer=True))
    add_labels(ax, total, total_label)
    plt.tight_layout()
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    print(f"Saved: '{filename}'")
    plt.show()

output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)

# ── 4 losse figuren ──────────────────────────────────────────────────────────
save_chart(
    bio_paper_series, 'Number of Papers',
    'Biobased / Biodegradable – Unique Articles',
    total_papers, 'papers', 'viridis',
    os.path.join(output_folder, 'bio_papers.png')
)

save_chart(
    bio_measure_series, 'Number of Measurements',
    'Biobased / Biodegradable – Total Measurements',
    total_bio_measurements, 'measurements', 'magma',
    os.path.join(output_folder, 'bio_measurements.png')
)

save_chart(
    ref_paper_series, 'Number of Papers',
    'Reference Materials – Unique Articles',
    total_papers, 'papers', 'viridis',
    os.path.join(output_folder, 'reference_papers.png')
)

save_chart(
```

```
    ref_measure_series, 'Number of Measurements',
    'Reference Materials – Total Measurements',
    total_ref_measurements, 'measurements', 'magma',
    os.path.join(output_folder, 'reference_measurements.png')
)
```

Dit slaat 4 losse bestanden op in `visualizations_v7`:

- `bio_papers.png`
- `bio_measurements.png`
- `reference_papers.png`
- `reference_measurements.png`

kan je hier weer een bladzijde van maken voor in mijn word bestand zodat ik die makkelijk kan kopieren?

31 Mar

Ran 3 commands, viewed 2 files, created a file

Claude's response was interrupted

nee doe het zelf, zet het in drive, ik hoef hier geen code van

31 Mar

Ran 2 commands

Connector search
10 connectors
Google Drive, AWS Marketplace, Box, Klaviyo, Cloudinary, Lumin, Lorikeet, Egnyte, Intuit QuickBooks, Slack

Je Google Drive is nog niet verbonden. Ik kan het bestand wel alvast downloaden — zodra je Drive koppelt kan ik het er direct naartoe sturen.

Presented file

Het Word-bestand staat klaar om te downloaden — de 4 figuren staan in een 2×2 raster op één liggende A4-pagina. Als je Drive wil koppelen kan ik het er de volgende keer direct naartoe uploaden!

For your drive
Beta

Google Drive
Connected

Looking for something else? Browse all connectors

liever in een verticale a4

31 Mar

Edited 2 files, ran 2 commands, read a file

Hier is hij — nu staande A4 met de 4 figuren in een 2×2 raster. De figuren zijn iets kleiner dan in de liggende versie maar passen netjes op één pagina.

Figures barrierrepresentativeness
Document · DOCX

kan je ook per figuur captions verzinnen en die hier in de chat zetten?

31 Mar

Op basis van de figuren:

bio_papers.png *Figure 1. Number of unique articles reporting WVTR, OTR, and CO_2TR measurements for biobased and/or biodegradable packaging films, expressed as a percentage of the total included studies (n = 19).*

---

bio_measurements.png *Figure 2. Total number of standardised barrier measurements for biobased and/or biodegradable packaging films per barrier property, expressed as a percentage of all biobased/biodegradable measurements in the dataset (n = 62).*

---

reference_papers.png *Figure 3. Number of unique articles reporting WVTR, OTR, and CO_2TR measurements for conventional reference polymers, expressed as a percentage of the total included studies (n = 19).*

---

reference_measurements.png *Figure 4. Total number of standardised barrier measurements for conventional reference polymers per barrier property, expressed as a percentage of all reference material measurements in the dataset (n = 61).*

Wil je de nummering anders, of de stijl meer formeel/academisch?

per grafiek, dus 4 captions

31 Mar

Dat zijn al 4 captions — één per figuur Wil je ze misschien anders geformuleerd, korter, of in het Nederlands?

gemini heeft ook dit een keer voor me gemaakt, wil jij hetzelfde doen maar dan voor deze dataset? import matplotlib.pyplot as plt import seaborn as sns import pandas as pd import numpy as np import os # 1. Prepare the data for the consolidated view # We define our target properties and their corresponding Temp/RH

columns prop_cols = { 'OTR': {'val': 'OTR_norm_100µm', 't': 'OTR_temperature_°C', 'rh': 'OTR_RH_%'}, 'CO2TR': {'val': 'CO2TR_norm_100µm', 't': 'CO2TR_temperature_°C', 'rh': 'CO2TR_RH_%'}, 'WVTR': {'val': 'WVTR_norm_100µm', 't': 'WVTR_temperature_°C', 'rh': 'WVTR_RH_%'} } # Create a list to store the processed data landscape_data = [] for prop, cols in prop_cols.items(): # Filter rows where this specific property has a value temp_df = df[df[cols['val']].notna()].copy()

Create a combined 'Condition' string for the X-axis # Using .map(str) to handle potential NaNs before joining temp_df['Condition'] = (temp_df[cols['t']].astype(str) + "°C / " + temp_df[cols['rh']].astype(str) + "% RH")

Remove cases where Temp or RH were 'nan' strings temp_df = temp_df[~temp_df['Condition'].str.contains('nan')]

Count occurrences and unique papers counts = temp_df.groupby('Condition').agg(Count=('Article_ID', 'count'), Papers=('Article_ID', 'nunique')).reset_index()

31 Mar

Hier is de aangepaste code voor jouw dataset:

python

```python
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
import os

# 1. Load & filter
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')
df = df[df['Included_after_screening'] == 'Yes'].copy()

plt.rcParams['font.family'] = 'Times New Roman'

# 2. Prepare data
prop_cols = {
   'OTR':   {'val': 'OTR100_cm3·m-2·d-1·bar-1',   't': 'OTR_temperature_°C',   'rh': 'OTR_RH_%'},
   'CO$_2$TR': {'val': 'CO2TR100_cm3·m-2·d-1·bar-1','t': 'CO2TR_temperature_°C', 'rh': 'CO2TR_RH_%'},
   'WVTR':  {'val': 'WVTR100_g·m-2·d-1 ',         't': 'WVTR_temperature_°C',  'rh': 'WVTR_RH_%'},
}

landscape_data = []
for prop, cols in prop_cols.items():
   temp_df = df[df[cols['val']].notna()].copy()
   temp_df['Condition'] = (
      temp_df[cols['t']].astype(str) + '°C / ' +
      temp_df[cols['rh']].astype(str) + '% RH'
   )
   temp_df = temp_df[~temp_df['Condition'].str.contains('nan')]

   counts = temp_df.groupby('Condition').agg(
      Count=('Article_ID', 'count'),
      Papers=('Article_ID', 'nunique')
   ).reset_index()

   total_prop_m = counts['Count'].sum()
   counts['Percentage'] = (counts['Count'] / total_prop_m) * 100
   counts['Property'] = prop
   landscape_data.append(counts)

summary_df = pd.concat(landscape_data)

# 3. Pivot
pivot_counts = summary_df.pivot(index='Property', columns='Condition', values='Count').fillna(0)
pivot_perc   = summary_df.pivot(index='Property', columns='Condition', values='Percentage').fillna(0)

pivot_counts = pivot_counts.reindex(['OTR', 'CO$_2$TR', 'WVTR'])
pivot_perc   = pivot_perc.reindex(['OTR', 'CO$_2$TR', 'WVTR'])

# 4. Annotation matrix
annot_matrix = []
for i in range(len(pivot_counts)):
   row = []
   for j in range(len(pivot_counts.columns)):
      count = pivot_counts.iloc[i, j]
      perc  = pivot_perc.iloc[i, j]
      row.append(f"{int(count)}\n({perc:.1f}%)" if count > 0 else "")
   annot_matrix.append(row)

# 5. Plot
plt.figure(figsize=(20, 4))
sns.set_theme(style='white')
plt.rcParams['font.family'] = 'Times New Roman'
```

```python
ax = sns.heatmap(
    pivot_counts,
    annot=np.array(annot_matrix),
    fmt='',
    cmap='YlGnBu',
    linewidths=0.5,
    cbar_kws={'label': 'Number of Measurements'},
    annot_kws={'size': 10, 'weight': 'bold'}
)

plt.title('Measurement Condition Landscape across Barrier Properties',
          fontsize=16, pad=25, fontweight='bold')
plt.xlabel('Condition (Temperature / Relative Humidity)', fontsize=12, labelpad=15)
plt.ylabel('Barrier Property', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()

# 6. Save
output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)
save_path = os.path.join(output_folder, 'consolidated_condition_landscape.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
print(f"Saved as '{save_path}'.")
plt.show()
```

Wijzigingen ten opzichte van het Gemini-script:

- Juiste kolomnamen voor jouw dataset
- Filter op `Included_after_screening == 'Yes'`
- `CO$_2$TR` voor subscript zonder font-warning
- `figsize=(20, 4)` want je hebt maar 3 rijen — iets compacter dan het origineel
- Opslaan in `visualizations_v7`

de tekst is wel een beetje klein als ik m op een a4tje plak, kan dat in verhouding tot het figuur wat groter?

31 Mar

Zet gewoon `annot_kws` en de andere fontsizes omhoog — hier de relevante regels om aan te passen:

python

```python
plt.figure(figsize=(20, 5))  # iets hogere figuur voor meer ruimte

ax = sns.heatmap(
    pivot_counts,
    annot=np.array(annot_matrix),
    fmt='',
    cmap='YlGnBu',
    linewidths=0.5,
    cbar_kws={'label': 'Number of Measurements'},
    annot_kws={'size': 14, 'weight': 'bold'}  # was 10
)

plt.title('Measurement Condition Landscape across Barrier Properties',
          fontsize=20, pad=25, fontweight='bold')  # was 16
plt.xlabel('Condition (Temperature / Relative Humidity)', fontsize=15, labelpad=15)  # was 12
plt.ylabel('Barrier Property', fontsize=15)  # was 12
plt.xticks(rotation=45, ha='right', fontsize=13)  # was default
plt.yticks(rotation=0, fontsize=13)  # was default
```

Als het nog steeds te klein is kun je `annot_kws={'size': 16}` en de andere fontsizes nog een stapje omhoog doen.

nu mis ik de legende

31 Mar

Voeg `cbar_kws` uit maar zorg dat de colorbar-label ook groter is:

python

```python
ax = sns.heatmap(
    pivot_counts,
    annot=np.array(annot_matrix),
    fmt='',
    cmap='YlGnBu',
    linewidths=0.5,
    cbar_kws={'label': 'Number of Measurements', 'shrink': 0.8},
    annot_kws={'size': 14, 'weight': 'bold'}
)

# Colorbar label groter maken
ax.collections[0].colorbar.ax.set_ylabel('Number of Measurements', fontsize=13)
ax.collections[0].colorbar.ax.tick_params(labelsize=12)
```

De colorbar was er al in de vorige versie, maar seaborn maakt de label standaard klein. Met `ax.collections[0].colorbar.ax.set_ylabel()` pas je die grootte apart aan.

kan je een caption schrijven voor in mijn report?

31 Mar

Figure 5. Heatmap showing the distribution of measurement conditions (temperature and relative humidity) across barrier properties (OTR, CO_2TR, and WVTR) for included studies. Each cell displays the number of measurements and the corresponding percentage of the total measurements for that barrier property. Empty cells indicate that no measurements were conducted under that condition.

Wil je het nummer aanpassen of iets anders benadrukt hebben?

dit heeft gemini ook een keer gemaakt. kan je dit nadoen? 1x voor biobased and biodegradable materials en 1x voor de reference materials: import plotly.express as px import pandas as pd import os # 1. Prepare the data total_papers = df['Article_ID'].nunique() # Count unique papers per Polymer_type and Material_family sun_df = df.groupby(['Material_family', 'Polymer_type']).agg(paper_count=('Article_ID', 'nunique')).reset_index() # Calculate percentages based on total papers sun_df['percentage'] = (sun_df['paper_count'] / total_papers) * 100 # 2. CREATE CUSTOM LABELS FOR SEGMENTS sun_df['display_label'] = (sun_df['Polymer_type'] + "
n=" + sun_df['paper_count'].astype(str) + " (" + sun_df['percentage'].round(1).astype(str) + "%)") # 3. Build the Sunburst with the capitalized title fig = px.sunburst(sun_df, path=['Material_family', 'display_label'], values='paper_count', color='Material_family', color_discrete_sequence=px.colors.qualitative.Safe, title="Distribution of Proposed Biobased and Biodegradable Polymers") # 4. Final Styling fig.update_traces(textinfo="label", insidetextorientation='radial', hovertemplate='%{label}') fig.update_layout(margin=dict(t=80, l=20, r=20, b=20), font=dict(size=14, family="Arial"), title_font=dict(size=20, family="Arial Black"), # Slightly bolder for the main title paper_bgcolor='white') # 5. Save the result output_folder =

31 Mar

Hier zijn beide versies:

Cel 1 – Biobased / Biodegradable materials

python

```python
import plotly.express as px
import pandas as pd
import os

# Load & filter
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')
df = df[df['Included_after_screening'] == 'Yes'].copy()
bio = df[(df['Biobased'] == 'Yes') | (df['Biodegradable'] == 'Yes')].copy()

total_papers = bio['Article_ID'].nunique()

sun_df = bio.groupby(['Material_family', 'Polymer_type']).agg(
    paper_count=('Article_ID', 'nunique')
).reset_index()
```

```python
sun_df['percentage'] = (sun_df['paper_count'] / total_papers) * 100
sun_df['display_label'] = (
    sun_df['Polymer_type'] + '<br>n=' +
    sun_df['paper_count'].astype(str) + ' (' +
    sun_df['percentage'].round(1).astype(str) + '%)'
)

fig = px.sunburst(
    sun_df,
    path=['Material_family', 'display_label'],
    values='paper_count',
    color='Material_family',
    color_discrete_sequence=px.colors.qualitative.Safe,
    title='Distribution of Biobased and Biodegradable Polymers'
)

fig.update_traces(
    textinfo='label',
    insidetextorientation='radial',
    hovertemplate='<b>%{label}</b>'
)
fig.update_layout(
    margin=dict(t=80, l=20, r=20, b=20),
    font=dict(size=14, family='Times New Roman'),
    title_font=dict(size=20, family='Times New Roman'),
    paper_bgcolor='white'
)

output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)
fig.write_html(os.path.join(output_folder, 'sunburst_biobased.html'))
fig.show()
```

Cel 2 – Reference materials

python

```python
ref = df[df['Reference_polymer'].notna()].copy()

total_papers_ref = ref['Article_ID'].nunique()

sun_ref = ref.groupby(['Material_family', 'Polymer_type']).agg(
    paper_count=('Article_ID', 'nunique')
).reset_index()

sun_ref['percentage'] = (sun_ref['paper_count'] / total_papers_ref) * 100
sun_ref['display_label'] = (
    sun_ref['Polymer_type'] + '<br>n=' +
    sun_ref['paper_count'].astype(str) + ' (' +
    sun_ref['percentage'].round(1).astype(str) + '%)'
)

fig2 = px.sunburst(
    sun_ref,
    path=['Material_family', 'display_label'],
    values='paper_count',
    color='Material_family',
    color_discrete_sequence=px.colors.qualitative.Safe,
    title='Distribution of Reference Polymers'
)

fig2.update_traces(
    textinfo='label',
    insidetextorientation='radial',
    hovertemplate='<b>%{label}</b>'
)
fig2.update_layout(
    margin=dict(t=80, l=20, r=20, b=20),
    font=dict(size=14, family='Times New Roman'),
    title_font=dict(size=20, family='Times New Roman'),
    paper_bgcolor='white'
)

fig2.write_html(os.path.join(output_folder, 'sunburst_reference.html'))
fig2.show()
```

Twee kanttekeningen:

- Voor de referentiematerialen gebruik ik `Reference_polymer` om de rijen te filteren — even checken of `Material_family` en `Polymer_type` daar ook ingevuld zijn, anders komen die segmenten leeg uit
- De output is `.html` (interactief), net als het Gemini-origineel. Wil je ook een statische `.png` versie om in Word te plakken?

ik had nog geen material family voor de reference polymers op zich is dat natuurlijk wel leuk om er bij te doen, kan je subcatergorien voor deze materialen bedenken? LDPE HDPE PP PET PS PVOH EVA PVC PVDC PA PVDC LDPE PE PVC PP HDPE LDPE EVOH (44% ethylene) PET EVOH (32% ethylene) NYLON 6 PET LDPE OPP OPP PE PS PVC PEN PA PVA EVOH PVDC

31 Mar

Hier zijn mijn voorgestelde subcategorieën:

Materiaal	Material_family
LDPE	Polyolefin
HDPE	Polyolefin
PP	Polyolefin
PE	Polyolefin
OPP	Polyolefin
PET	Polyester
PEN	Polyester
PS	Polystyrene
PVC	Vinyl polymer
PVDC	Vinyl polymer
PVOH / PVA	Vinyl polymer
EVOH	Vinyl polymer
EVA	Vinyl polymer
PA / Nylon 6	Polyamide

Logica:

- **Polyolefin** — koolstof-koolstof ruggengraat, geen polaire groepen (LDPE, HDPE, PP, PE, OPP)
- **Polyester** — esterverbinding in ruggengraat (PET, PEN)
- **Vinyl polymer** — vinylmonomeer als basis, inclusief copolymeren (PVC, PVDC, PVOH/PVA, EVOH, EVA)
- **Polyamide** — amideverbinding in ruggengraat (PA, Nylon 6)

Wil je dit zo gebruiken, of wil je bijvoorbeeld EVOH en PVOH apart zetten als **Barrier polymer** omdat ze specifiek om hun barrièreeigenschappen worden ingezet?

wat betekent dit voor hun eigenschappen? zijn ze daardoor ook meer beinvloedbaar door RH?

31 Mar

Ja, absoluut — de materiaalfamilie bepaalt sterk hoe gevoelig een polymeer is voor vocht, en dat heeft directe gevolgen voor de barrière-eigenschappen.

Polyolefinen (LDPE, HDPE, PP, PE, OPP) Volledig apolair — ze absorberen vrijwel geen water. Daardoor zijn hun OTR en CØTR nauwelijks RH-afhankelijk. WVP is inherent laag. Ze zijn juist goed als vochtbarrière, maar relatief slecht als zuurstofbarrière.

Polyesters (PET, PEN) Matig polair door de estergroepen. Beperkte RH-gevoeligheid — ze absorberen een kleine hoeveelheid vocht, maar dit heeft weinig effect op de OTR. PET heeft van nature al een goede zuurstofbarrière die stabiel blijft bij hogere luchtvochtigheid.

Vinyl polymeren — en hier zit het grote verschil:

- **PVC en PVDC** — relatief apolair, weinig RH-gevoelig, goede barrière-eigenschappen die stabiel blijven
- **EVOH** — dit is het klassieke voorbeeld van een polymeer dat extreem gevoelig is voor RH. Bij droge omstandigheden (0% RH) heeft EVOH een uitstekende zuurstofbarrière, maar bij hogere luchtvochtigheid neemt de OTR sterk toe omdat water de polaire OH-groepen plastificeert en de polymeerketens meer bewegingsruimte geeft. Dit is precies waarom EVOH altijd tussen andere lagen wordt gelamineerd in meerlaagse verpakkingen.
- **PVOH/PVA** — nog hydrofieler dan EVOH, lost zelfs op in water. Vrijwel onbruikbaar als barrière bij hoge RH zonder coating of laminering.
- **EVA** — minder polair dan EVOH, matige RH-gevoeligheid

Polyamiden (PA, Nylon 6) Polair door de amidegroepen — absorberen merkbaar vocht. Dit plastificeert het materiaal en verhoogt de OTR bij hogere RH, vergelijkbaar als bij EVOH maar minder extreem. PA wordt vaak gecombineerd met PE of PP om dit te compenseren.

Voor jouw dataset is dit relevant omdat je bij EVOH, PVOH en PA de metingen bij 0% RH en 50-53% RH waarschijnlijk sterk van elkaar ziet afwijken — terwijl je bij polyolefinen nauwelijks verschil verwacht. Dit verklaart ook waarom de referentiemetingen zo verspreid zijn over verschillende RH-condities: onderzoekers meten EVOH bewust droog om de theoretische maximale barrièrewaarde te krijgen.

KAN JE OOK NOG EEN W=VERSCHIL MAKEN TUSSEN ALIPHATIC EN AROMATIC POLYESTER?

31 Mar

Ja zeker! PET en PEN zijn allebei **aromatisch** (benzeenring in de ruggengraat), maar het onderscheid is relevant:

Materiaal	Material_family
PET	Aromatic polyester
PEN	Aromatic polyester

Er zijn geen **aliphatische** polyesters in jouw referentielijst — die komen juist veel voor bij de **biobased** materialen (PBS, PLA, PBAT). Dus voor de referentiematerialen specifiek is het onderscheid niet nodig, maar het is wel zinvol om in je thesis te benoemen waarom:

Aromatische polyesters hebben door de rigide benzeenring een veel dichtere pakking van de polymeerketens, wat resulteert in een betere barrière tegen zuurstof en CO_2 vergeleken met aliphatische polyesters. PLA bijvoorbeeld is aliphatisch en heeft daardoor een significant hogere OTR dan PET bij vergelijkbare dikte. Dit is een van de centrale uitdagingen voor biobased verpakkingen — de meeste biobased polyesters zijn aliphatisch en halen de barrièreperformance van PET nog niet zonder additieven of coatings.

Wil je dat ik de volledige bijgewerkte Material_family tabel geef inclusief dit onderscheid, zodat je die direct in je dataset kunt invoeren?

kan je hier de family ook specifieker maken> pbs is dus blijkbaar een alifatische biopolyester: PLA Biosynthetic polymer PLA Biosynthetic polymer PVA Biosynthetic polymer PBS Biosynthetic polymer Furanoate polyester Biosynthetic polymer PHBV Biosynthetic polymer PBSA Biosynthetic polymer Furanoate polyester Biosynthetic polymer Furanoate polyester Biosynthetic polymer PLA Biosynthetic polymer PLA Biosynthetic polymer PLA Biosynthetic polymer PBS Biosynthetic polymer PBS Biosynthetic polymer

31 Mar

Hier is de bijgewerkte indeling:

Polymer_type	Material_family (oud)	Material_family (nieuw)
PLA	Biosynthetic polymer	Aliphatic polyester
PBS	Biosynthetic polymer	Aliphatic polyester
PBSA	Biosynthetic polymer	Aliphatic polyester
PHBV	Biosynthetic polymer	Aliphatic polyester
Furanoate polyester	Biosynthetic polymer	Aromatic polyester
PVA	Biosynthetic polymer	Vinyl polymer

Toelichting:

- **PLA, PBS, PBSA, PHBV** — allemaal aliphatische polyesters, geen aromatische ring in de ruggengraat
- **Furanoate polyester (PEF)** — heeft een furaanring in plaats van een benzeenring, wat hem aromatisch maakt. Interessant voor jouw thesis want PEF wordt juist gezien als biobased alternatief voor PET met vergelijkbare of betere barrière-eigenschappen
- **PVA** — is eigenlijk geen polyester maar een vinylpolymeer, net als bij de referentiematerialen — dus logisch om daar ook onder te zetten

Wil je dat ik ook de volledige gecombineerde tabel maak met zowel de biobased als de referentiematerialen in één overzicht, zodat je die consistent kunt gebruiken in je dataset?

dit is weer een update van mn bestand, hier gaan we mee verder met de visualisaties. kunnen we hier weer die sunburst van net mee maken?

31 Mar

Ran 3 commands

Top, alles staat al netjes in het bestand! Hier is de code:

python

```python
import plotly.express as px
import pandas as pd
import os

# Load & filter
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')
df = df[df['Included_after_screening'] == 'Yes'].copy()

output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)


# ── Figuur 1: Biobased / Biodegradable ──────────────────────────────────────
bio = df[(df['Biobased'] == 'Yes') | (df['Biodegradable'] == 'Yes')].copy()
total_bio = bio['Article_ID'].nunique()

sun_bio = bio.groupby(['Material_family', 'Polymer_type']).agg(
    paper_count=('Article_ID', 'nunique')
).reset_index()

sun_bio['percentage'] = (sun_bio['paper_count'] / total_bio) * 100
sun_bio['display_label'] = (
    sun_bio['Polymer_type'] + '<br>n=' +
    sun_bio['paper_count'].astype(str) + ' (' +
    sun_bio['percentage'].round(1).astype(str) + '%)'
)

fig1 = px.sunburst(
    sun_bio,
    path=['Material_family', 'display_label'],
    values='paper_count',
    color='Material_family',
    color_discrete_sequence=px.colors.qualitative.Safe,
    title='Distribution of Biobased and Biodegradable Polymers'
)
fig1.update_traces(
    textinfo='label',
    insidetextorientation='radial',
    hovertemplate='<b>%{label}</b>'
)
fig1.update_layout(
    margin=dict(t=80, l=20, r=20, b=20),
    font=dict(size=14, family='Times New Roman'),
    title_font=dict(size=20, family='Times New Roman'),
    paper_bgcolor='white'
)
fig1.write_html(os.path.join(output_folder, 'sunburst_biobased.html'))
fig1.show()


# ── Figuur 2: Reference materials ────────────────────────────────────────────
ref = df[df['Reference_polymer'].notna()].copy()
total_ref = ref['Article_ID'].nunique()

sun_ref = ref.groupby(['Reference_polymer_family', 'Reference_polymer']).agg(
    paper_count=('Article_ID', 'nunique')
).reset_index()

sun_ref['percentage'] = (sun_ref['paper_count'] / total_ref) * 100
sun_ref['display_label'] = (
    sun_ref['Reference_polymer'] + '<br>n=' +
    sun_ref['paper_count'].astype(str) + ' (' +
    sun_ref['percentage'].round(1).astype(str) + '%)'
)

fig2 = px.sunburst(
    sun_ref,
    path=['Reference_polymer_family', 'display_label'],
    values='paper_count',
    color='Reference_polymer_family',
    color_discrete_sequence=px.colors.qualitative.Safe,
    title='Distribution of Reference Polymers'
)
fig2.update_traces(
    textinfo='label',
    insidetextorientation='radial',
    hovertemplate='<b>%{label}</b>'
)
fig2.update_layout(
    margin=dict(t=80, l=20, r=20, b=20),
    font=dict(size=14, family='Times New Roman'),
    title_font=dict(size=20, family='Times New Roman'),
```

```
    paper_bgcolor='white'
)
fig2.write_html(os.path.join(output_folder, 'sunburst_reference.html'))
fig2.show()
```

De referentiefiguur gebruikt nu `Reference_polymer_family` en `Reference_polymer` die al in je dataset staan — geen extra mapping nodig. Beide worden opgeslagen als interactieve HTML in `visualizations_v7`.

ik wil de sunburst eigenlijk gewoon als afbeelding kopieren in mn word

31 Mar

Dan kun je de HTML openen in je browser en daar een screenshot van maken, maar netter is dit — voeg onderaan elke figuur toe:

python

```
fig1.write_image(os.path.join(output_folder, 'sunburst_biobased.png'),
            width=800, height=800, scale=3)

fig2.write_image(os.path.join(output_folder, 'sunburst_reference.png'),
            width=800, height=800, scale=3)
```

`scale=3` zorgt voor hoge resolutie (300 dpi equivalent). Je hebt wel de `kaleido` library nodig:

python

```
pip install kaleido
```

Daarna werkt `write_image` direct.

uhmmm kan je me de hele code anders geven=

31 Mar

Natuurlijk!

python

```
import plotly.express as px
import pandas as pd
import os

# Load & filter
file_name = 'ThesisSharondeBondt_SystematicReview_RawData_v7_20260303.xlsx'
df = pd.read_excel(file_name, sheet_name='Data')
df = df[df['Included_after_screening'] == 'Yes'].copy()

output_folder = 'visualizations_v7'
os.makedirs(output_folder, exist_ok=True)


# ── Figuur 1: Biobased / Biodegradable ──────────────────────────────────
bio = df[(df['Biobased'] == 'Yes') | (df['Biodegradable'] == 'Yes')].copy()
total_bio = bio['Article_ID'].nunique()

sun_bio = bio.groupby(['Material_family', 'Polymer_type']).agg(
    paper_count=('Article_ID', 'nunique')
).reset_index()

sun_bio['percentage'] = (sun_bio['paper_count'] / total_bio) * 100
sun_bio['display_label'] = (
    sun_bio['Polymer_type'] + '<br>n=' +
    sun_bio['paper_count'].astype(str) + ' (' +
    sun_bio['percentage'].round(1).astype(str) + '%)'
)
```

```
fig1 = px.sunburst(
    sun_bio,
    path=['Material_family', 'display_label'],
    values='paper_count',
    color='Material_family',
    color_discrete_sequence=px.colors.qualitative.Safe,
    title='Distribution of Biobased and Biodegradable Polymers'
)
fig1.update_traces(
    textinfo='label',
    insidetextorientation='radial',
    hovertemplate='<b>%{label}</b>'
)
fig1.update_layout(
    margin=dict(t=80, l=20, r=20, b=20),
    font=dict(size=14, family='Times New Roman'),
    title_font=dict(size=20, family='Times New Roman'),
    paper_bgcolor='white'
)
fig1.write_image(os.path.join(output_folder, 'sunburst_biobased.png'),
                 width=800, height=800, scale=3)
fig1.show()


# ── Figuur 2: Reference materials ──────────────────────────────────────────
ref = df[df['Reference_polymer'].notna()].copy()
total_ref = ref['Article_ID'].nunique()

sun_ref = ref.groupby(['Reference_polymer_family', 'Reference_polymer']).agg(
    paper_count=('Article_ID', 'nunique')
).reset_index()

sun_ref['percentage'] = (sun_ref['paper_count'] / total_ref) * 100
sun_ref['display_label'] = (
    sun_ref['Reference_polymer'] + '<br>n=' +
    sun_ref['paper_count'].astype(str) + ' (' +
    sun_ref['percentage'].round(1).astype(str) + '%)'
)

fig2 = px.sunburst(
    sun_ref,
    path=['Reference_polymer_family', 'display_label'],
    values='paper_count',
    color='Reference_polymer_family',
    color_discrete_sequence=px.colors.qualitative.Safe,
    title='Distribution of Reference Polymers'
)
fig2.update_traces(
    textinfo='label',
    insidetextorientation='radial',
    hovertemplate='<b>%{label}</b>'
)
fig2.update_layout(
    margin=dict(t=80, l=20, r=20, b=20),
    font=dict(size=14, family='Times New Roman'),
    title_font=dict(size=20, family='Times New Roman'),
    paper_bgcolor='white'
)
fig2.write_image(os.path.join(output_folder, 'sunburst_reference.png'),
                 width=800, height=800, scale=3)
fig2.show()
```

Vergeet niet eerst `kaleido` te installeren als je dat nog niet hebt:

```
pip install kaleido
```

```
SyntaxError: invalid syntax
```

31 Mar

Verwijder de drie backticks — die zijn voor de chatweergave en horen niet in je Jupyter cel. Dus niet:

```
pip install kaleido
```

Maar in een aparte Jupyter cel:

python

```
!pip install kaleido
```

Het uitroepteken `!` is belangrijk — daarmee voer je terminal-commando's uit vanuit Jupyter.

hmmm laat maar, maar ik wil wel dat de titel recht boven de figuren te zien is

31 Mar

Voeg `title_x` toe aan `update_layout`:

python

```
fig1.update_layout(
    margin=dict(t=80, l=20, r=20, b=20),
    font=dict(size=14, family='Times New Roman'),
    title_font=dict(size=20, family='Times New Roman'),
    title_x=0.5,
    paper_bgcolor='white'
)
```

Hetzelfde voor `fig2`. De `title_x=0.5` centreert de titel boven de figuur.

kun je captiosn voor de 2 figuren schrijven voor in mn report=

31 Mar

Figuur 1 – Biobased / Biodegradable *Figure X. Sunburst diagram showing the distribution of biobased and biodegradable polymers across material families included in this systematic review. The inner ring represents material families and the outer ring represents individual polymer types. Values indicate the number of unique articles (n) and the percentage of the total included studies.*

Figuur 2 – Reference materials *Figure X. Sunburst diagram showing the distribution of conventional reference polymers across material families reported in the included studies. The inner ring represents material families and the outer ring represents individual polymer types. Values indicate the number of unique articles (n) and the percentage of the total included studies.*

ik zou ook wel een scatterplot willen hebben per transmission rate (wvtr, otr, co2tr) waarin per materiaal deze wordt weer gegeven, met de waarden van de reference materials als reference line. van de biobased/biodegradable materials wil ik deze alleen vergelijken als de RH en T tussen de materialen hetzelfde is (en dan de 3 meest vertegenwoordigde meet condities per barrier property) voor de reference line: polyolefins mogen sws worden weer gegeven, polyesters en de rest ook alleen als de rh hetzelfde is als die van de biopolymers

31 Mar

Laat me eerst de data goed bekijken voordat ik de code schrijf.

Ran 2 commands

Goed beeld van de data. De top 3 condities voor WVTR zijn 25°C/75%, 23°C/50%, en 25°C/60%. Voor OTR zijn er maar 2 condities (23°C/0% en 25°C/53%). Voor de referentielijnen: bij WVTR hebben de Polyolefins OPP bij 25°C/75% — dat matcht mooi. Hier is de volledige code: